Assignment Frontend Developer- UIDAI Innovation Vertical

Context: You are interviewing for the Frontend Developer role for the UIDAI Sandbox. This assignment tests your ability to build secure, accessible, and performant user interfaces for complex digital identity workflows using modern web technologies.

Instructions:

- Submit a single PDF named FrontendDev_[YourName]_Assignment.pdf.
- Include a link to a public GitHub repository containing your code for all three prompts.
- You may use React JS ,Angular , HTML5,CSS3 with JavaScript/TypeScript. Keep written answers brief and use mock data for APIs.

The Scenario: Verifiable Credential Wallet PWA

A startup in the Sandbox wants to test a "Digital Wallet" experience. We need to build a Progressive Web App (PWA) frontend where a test user can log in, view their simulated identity credentials (like a mock Aadhaar or eKYC profile), and securely reveal sensitive data.

Part 1: Build the Dashboard & State Management (Coding)

Write the frontend code for the main Credential Dashboard view.

- **The UI:** Build a responsive grid or list displaying a user's digital credentials.
- **State Management:** Use a state management library (like Redux, Context API, or MobX) to fetch and store a mock array of credential objects from a dummy API (GET /api/credentials).
- **Resilience:** Your code must explicitly handle and display three UI states: Loading, Error (e.g., if the mock API fails), and Empty (if the user has no credentials).

Part 2: Secure "Reveal" Component & Accessibility (Coding)

Identity data is highly sensitive. Write the code for a reusable SecureDataMask component used within the dashboard.

- **The Logic:** The component receives a sensitive string (e.g., a mock Aadhaar number). By default, it renders as masked (e.g., XXXX-XXXX-1234).
- **The Interaction:** It must include a "Tap to Reveal" button. Once clicked, the full string is displayed. **Crucially, write a timeout function that automatically re-masks the data after 10 seconds** and clears the unmasked value from the local component state.
- **Accessibility (a11y):** Your component code must be fully accessible (WCAG compliant). Include the necessary ARIA attributes, ensure the "Reveal" button is keyboard-navigable, and ensure screen readers announce the state change when the data is revealed or hidden.

Part 3: Offline Capabilities & Automated Testing (Coding)

Our Sandbox PWA must be resilient to network drops, and our security features must be heavily tested.

- **PWA Offline Support:** Write a specific Service Worker code snippet (using Workbox or vanilla JS) that intercepts the GET /api/credentials request. If the network is offline, it should serve the last known credential list from the browser's Cache Storage or IndexedDB.
- **Automated Testing:** We cannot risk the security timeout failing in production. Write a test suite using Jest and React Testing Library (or equivalent Angular testing tools) that explicitly mounts your SecureDataMask component, simulates a user click, verifies the text is unmasked, fast-forwards the timers by 10 seconds, and asserts that the text is securely masked again.